

Answers:

1. Convert the point (3, 3, 5) to homogeneous coordinates.
 - a. (3, 3, 5, 1)
Since this is a point, and not a direction, we would insert 1 for the w coordinate to allow translations to occur.
2. Convert the direction vector (0.707, 0.707, 0) to homogenous coordinates.
 - a. (0.707, 0.707, 0, 0)
Since this is a direction, not a point, we would insert 0 for the w coordinate to zero out any translational transforms.
3. What is the effect of setting 1 in the w coordinate?
 - a. Remember that the w coordinate controls how the translation affects that vector when multiplied by a 4x4 transformation matrix. A value of 1 specifies a point in 3D space, and will be fully affected by translation. A value of 0 is used for directions since it allows for no translation, but will still be affected by rotation and scale.
4. What is the effect of setting 0 in the w coordinate?
 - a. See answer to previous question.
5. Build a 2x2 matrix that scales along X by 2 and Y by 3.
 - a. $\begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}$
6. Build a 3x3 matrix that scales a homogeneous coordinate along X and Y by 3.
 - a. $\begin{pmatrix} 2 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 1 \end{pmatrix}$
7. Build a 3x3 matrix that scales a homogenous coordinate along X and Y by a constant scaling factor S.
 - a. $\begin{pmatrix} S & 0 & 0 \\ 0 & S & 0 \\ 0 & 0 & 1 \end{pmatrix}$
8. Build a 3x3 matrix that translates a homogeneous coordinate along X and Y by some 2D vector T.
 - a. $\begin{pmatrix} 1 & 0 & T.x \\ 0 & 1 & T.y \\ 0 & 0 & 1 \end{pmatrix}$
9. How would you apply a rotation matrix to a vector? Show an example.
 - a. To rotate a vector $\vec{v}$, you just need to multiply it by the rotation matrix R.
So $\vec{v'} = R\vec{v}$

Example: Given $R = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$ and $\vec{v} = (1, 0)$

$$\vec{v'} = R\vec{v} = (0, 1)$$

10. Explain how to combine two rotation matrices to achieve a composite rotation.
- You multiply the matrices together. So given rotation matrices R_1 and R_2 , you multiply them to get a composite rotation $R_1 R_2$.
11. Why is it important to apply scale first, then rotation, then translation in that specific order? Show an example with an incorrect order using a "model" with 2 points (0, 0), and (1, 0).
- Let's define our TRS as a translation of (2, 0), rotation of 90° , and a scale of 2. So applying them in the correct order:
 - After scale: (0, 0), (2, 0)
 - After Rotation: (0, 0), (0, 2)
 - After Translation: (2, 0), (2, 2)
 - Now let's do it wrong, same TRS, but different order.
 - After Scale: (0, 0), (2, 0)
 - After Translation: (2, 0), (4, 0)
 - After Rotation: (0, 2), (0, 4)
12. Provide a 4x4 identity matrix.
- $$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$
13. Why do games often store position, rotation, and scale in code separately rather than as a single matrix?
- Generally, multiple good reasons.
 - The first being that by keeping them as separate values, it allows you to operate on them in a straightforward manner. It's straightforward for other coders, since it's easy to read and work with. When you combine them into a single TRS matrix, the rotational components and the scale components are combined, so it can be difficult to extract out the individual rotations that led to that final rotation, for example.
 - Secondly, allowing you to operate on the matrix itself may result in invalid TRS matrices, since a 4x4 affine transform can represent many more linear transformations, but not all of which are desirable in a game setting and may lead to bugs.
 - Finally, it's more memory efficient to store them as separate values and combine them for the graphics pipeline.
14. What is an affine transformation?
- An affine transformation is a linear transformation followed by a translation. Basically, given some matrix RS , a vector v , and a translation vector t , the affine transformation is $RS * v + t$